

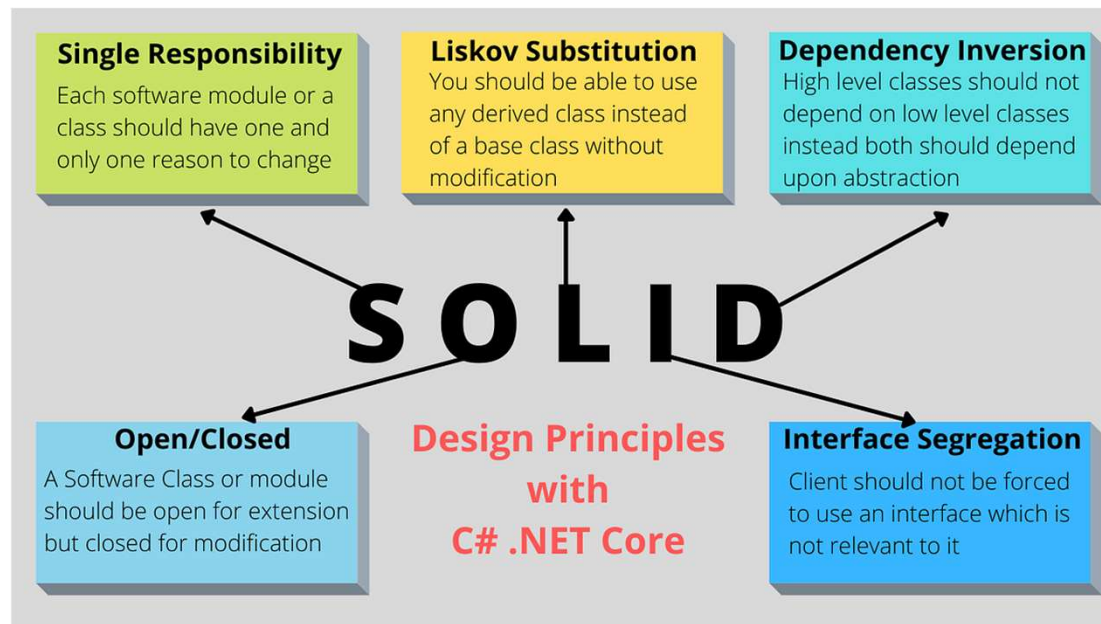


Modern Software Engineering & Architecture – Detailed Guide

SOLID, DRY, CLOUD, DEVOPS, MICROSERVICES,
DOCKER & KUBERNETES

SOLID Principles – Overview

- ▶ Five object-oriented design principles for maintainable software
- ▶ Improve scalability, flexibility, and testability
- ▶ Reduce tight coupling between components
- ▶ Widely used in enterprise and microservices systems
- ▶ Foundation for clean architecture



Single Responsibility Principle (SRP)

- ▶ A class should have only one reason to change
- ▶ Focus on one responsibility per class
- ▶ Reduces complexity and side effects
- ▶ Improves readability and unit testing
- ▶ Example: Invoice logic separated from printing and saving

Open / Closed Principle (OCP)

- ▶ Open for extension but closed for modification
- ▶ New functionality added without changing existing code
- ▶ Achieved using interfaces and abstraction
- ▶ Prevents regression bugs
- ▶ Example: Adding new payment method via interface

Liskov Substitution Principle (LSP)

- ▶ Derived classes must be substitutable for base classes
- ▶ Child class should not change expected behavior
- ▶ Ensures correctness in polymorphism
- ▶ Violations lead to runtime issues
- ▶ Example: Square vs Rectangle problem

Interface Segregation Principle (ISP)

- ▶ Clients should not depend on unused methods
- ▶ Split large interfaces into smaller ones
- ▶ Improves API usability
- ▶ Encourages role-based design
- ▶ Example: Printer, Scanner, Fax interfaces

Dependency Inversion Principle (DIP)

- ▶ High-level modules should not depend on low-level modules
- ▶ Both should depend on abstractions
- ▶ Promotes loose coupling
- ▶ Simplifies unit testing
- ▶ Example: Service depends on repository interface

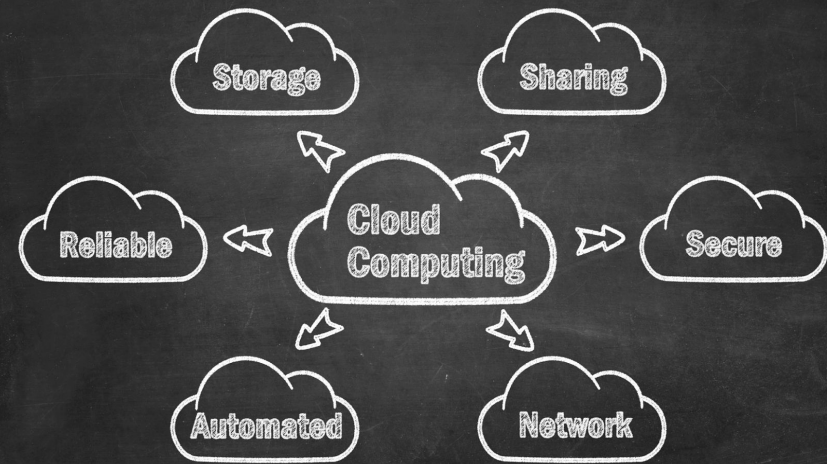
DRY Principle – Don't Repeat Yourself

- ▶ Each piece of knowledge should have a single source of truth
- ▶ Avoid duplication in code and configuration
- ▶ Simplifies maintenance
- ▶ Reduces bugs and inconsistencies
- ▶ Common in layered and microservices architecture



Cloud Computing – Fundamentals

- ▶ Delivery of computing services over the internet
- ▶ Includes servers, storage, databases, and networking
- ▶ Highly scalable and elastic
- ▶ Pay-as-you-use cost model
- ▶ Eliminates infrastructure management overhead





Cloud Service Models

- ▶ SaaS: Software delivered over browser
- ▶ PaaS: Platform for application development
- ▶ IaaS: Virtualized infrastructure resources
- ▶ Each model offers different control levels
- ▶ Supports different business needs

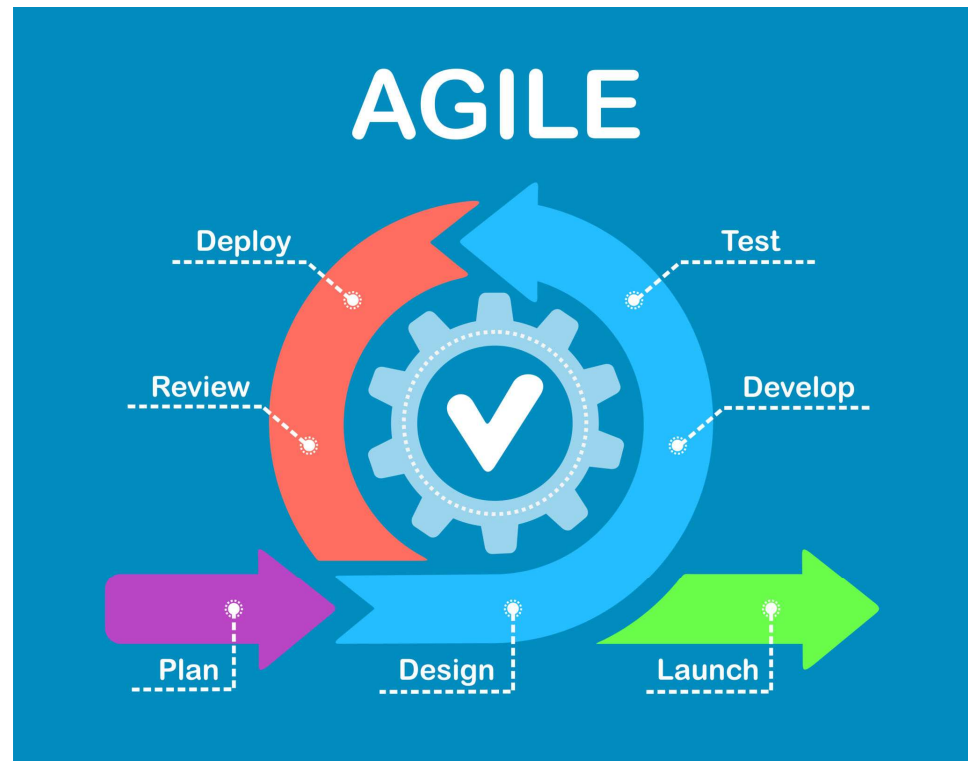
SaaS, PaaS, IaaS – Examples

- ▶ SaaS: Gmail, Salesforce
- ▶ PaaS: Heroku, Google App Engine
- ▶ IaaS: AWS EC2, Azure Virtual Machines
- ▶ Used by startups and enterprises



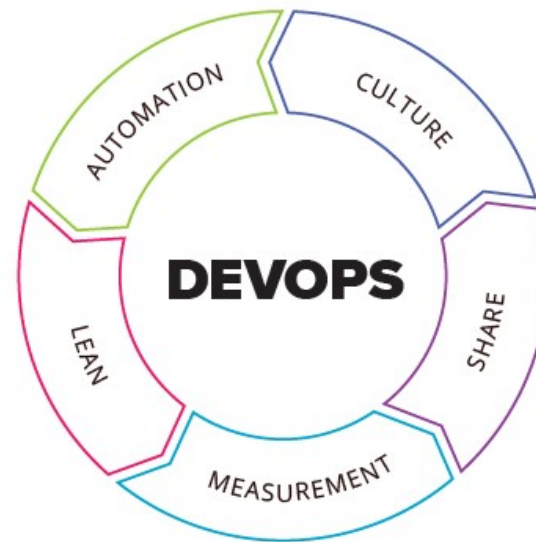
Agile Methodology

- ▶ Iterative and incremental development
- ▶ Customer collaboration and feedback
- ▶ Responds to changing requirements
- ▶ Improves delivery speed
- ▶ Scrum and Kanban frameworks



DevOps Principles

- ▶ Collaboration between Development and Operations
- ▶ Automation of build and deployment
- ▶ Continuous monitoring and feedback
- ▶ Improves reliability
- ▶ Enables faster releases



CI/CD Pipeline

- ▶ Continuous Integration: frequent code merges
- ▶ Automated build and testing
- ▶ Continuous Delivery and Deployment
- ▶ Reduces manual errors
- ▶ Tools: Jenkins, GitHub Actions, GitLab CI

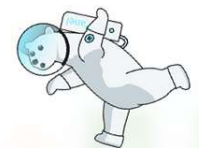
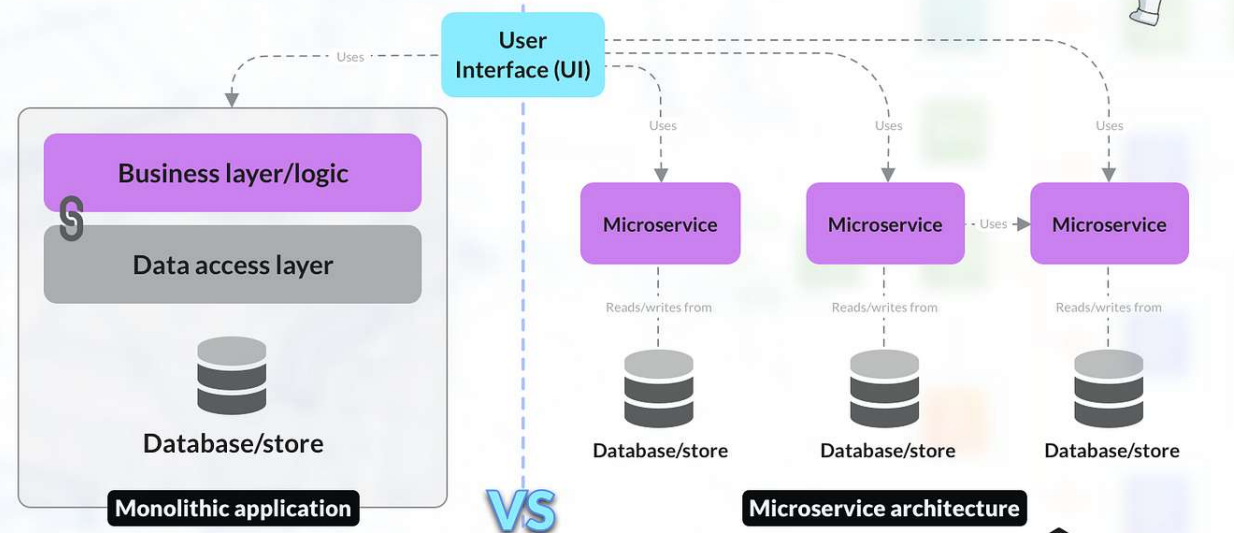
CI CD Pipeline



Microservices Architecture

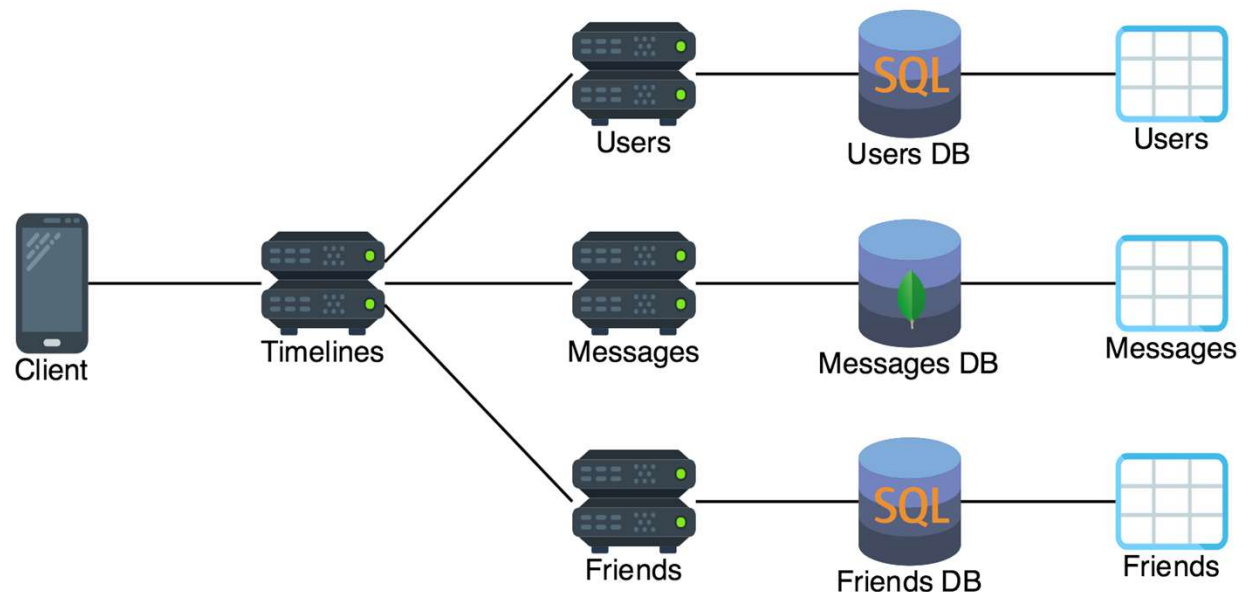
- ▶ Application split into small independent services
- ▶ Each service mapped to business capability
- ▶ Services communicate via APIs
- ▶ Fault isolation and scalability
- ▶ Ideal for large systems

Monolithic vs microservice architecture



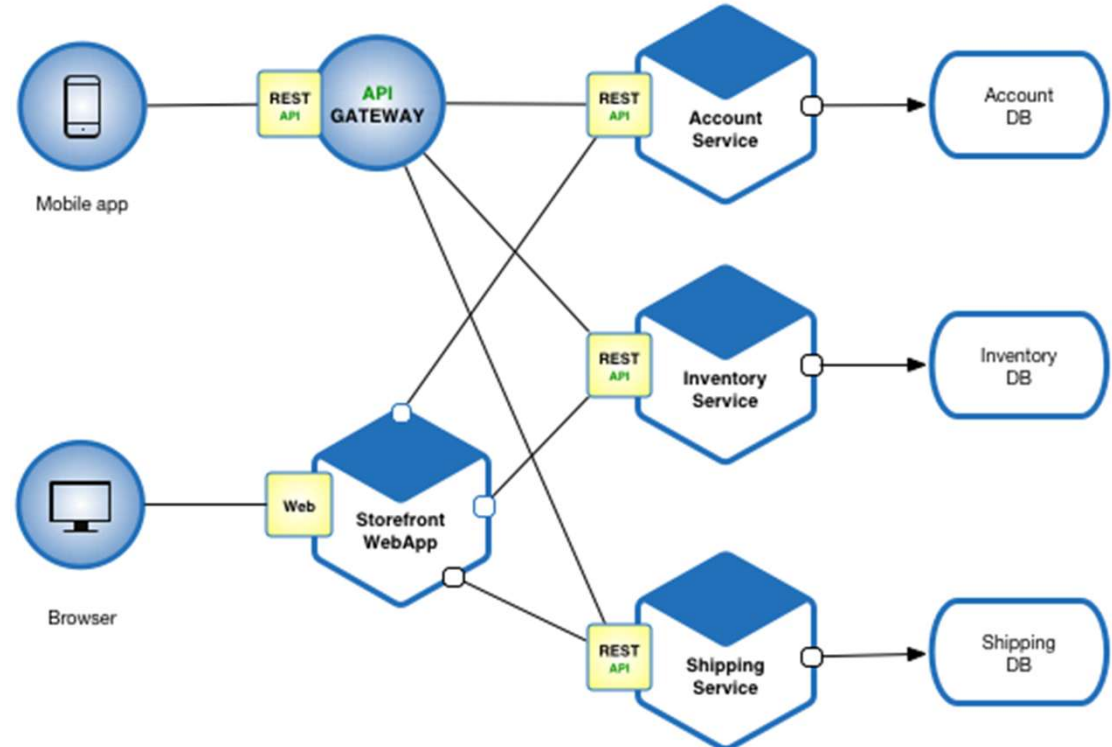
Decentralized Data Management

- ▶ Each service owns its database
- ▶ No shared database
- ▶ Ensures loose coupling
- ▶ Improves scalability
- ▶ Supports autonomy



Independent Deployment

- ▶ Each service deployed independently
- ▶ Faster release cycles
- ▶ Reduced deployment risk
- ▶ Supports CI/CD
- ▶ Minimizes downtime

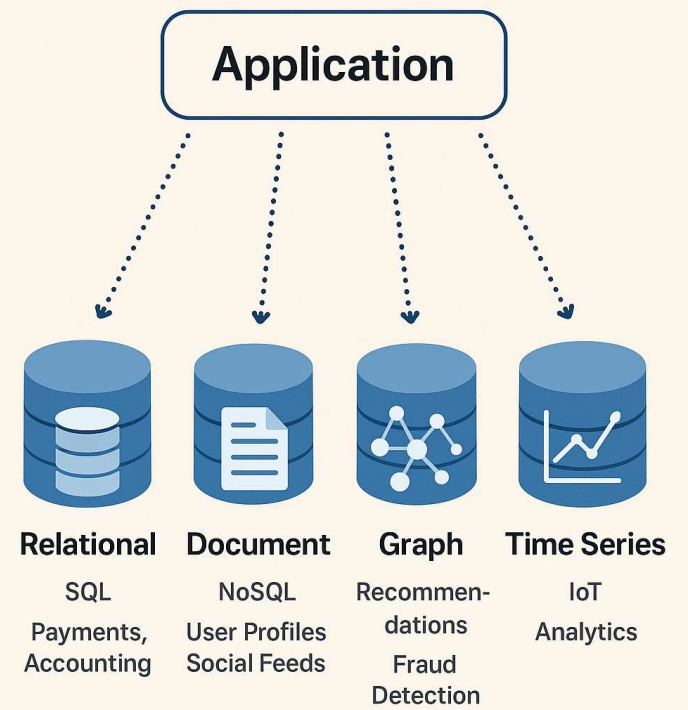


Polyglot Persistence & Programming

- ▶ Different databases for different services
- ▶ Choice based on service needs
- ▶ Supports SQL and NoSQL
- ▶ Multiple programming languages allowed
- ▶ Improves flexibility

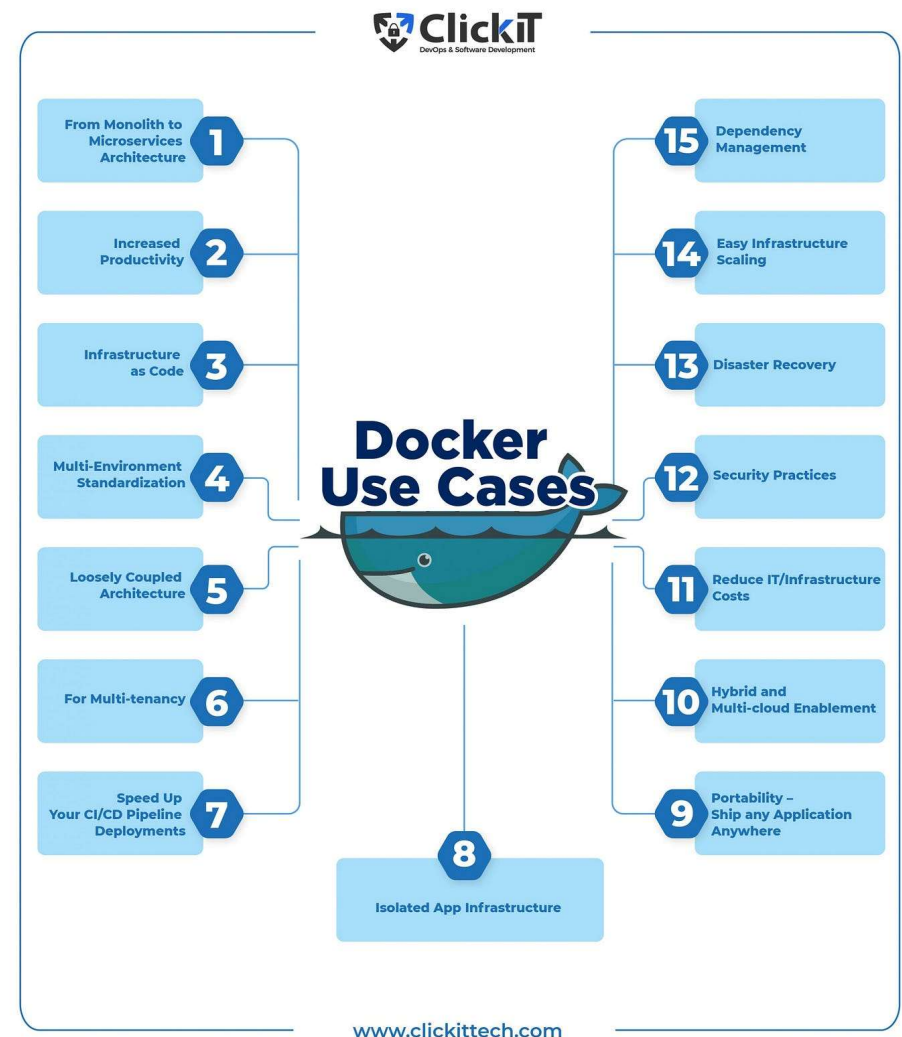
POLYGLOT PERSISTENCE

USING MULTIPLE DATA STORAGE SYSTEMS



Docker & Containerization

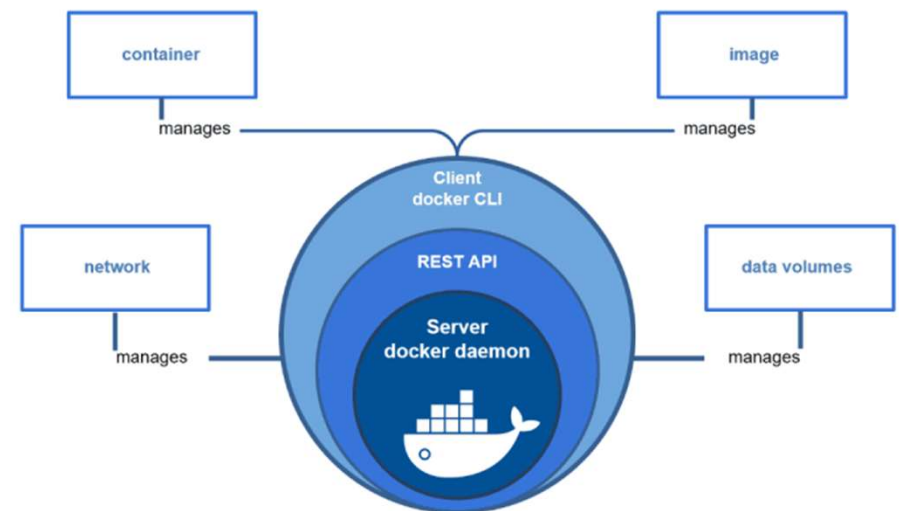
- ▶ Docker packages applications with dependencies
- ▶ Ensures environment consistency
- ▶ Lightweight compared to virtual machines
- ▶ Fast startup times
- ▶ Core tool in DevOps



Docker Components

- ▶ Docker Image: Application blueprint
- ▶ Docker Container: Running instance
- ▶ Docker file: Image build instructions
- ▶ Docker Engine: Runtime
- ▶ Portable and reusable

DOCKER COMPONENTS





Docker Hub & Registries

- ▶ Docker Hub stores public images
- ▶ Supports image versioning
- ▶ Private registries for enterprises
- ▶ Integrates with CI/CD
- ▶ Ensures secure distribution

Kubernetes – Overview

- ▶ Container orchestration platform
- ▶ Manages deployment and scaling
- ▶ Industry standard
- ▶ Originally developed by Google
- ▶ Works with Docker

KUBERNETES SYSTEM



Deployment



Scaling



Dev Ops Integration



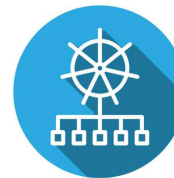
Pods/containers



Open Source



Management



Services



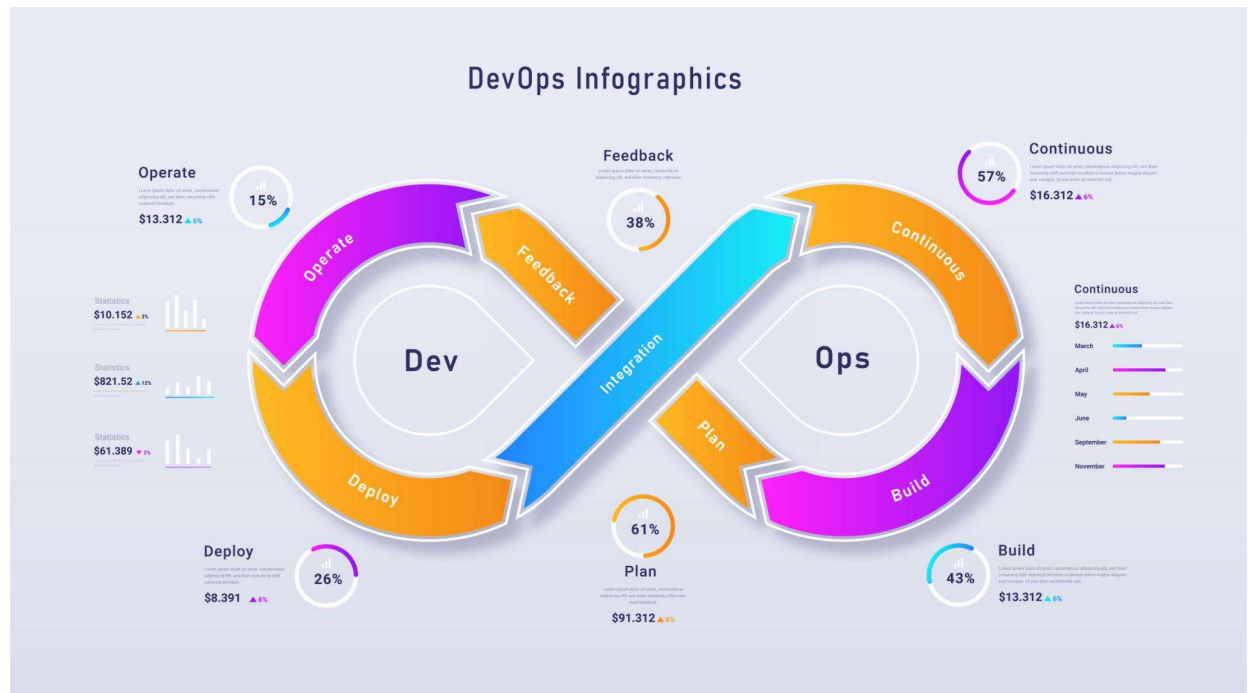
Container Management



Performance

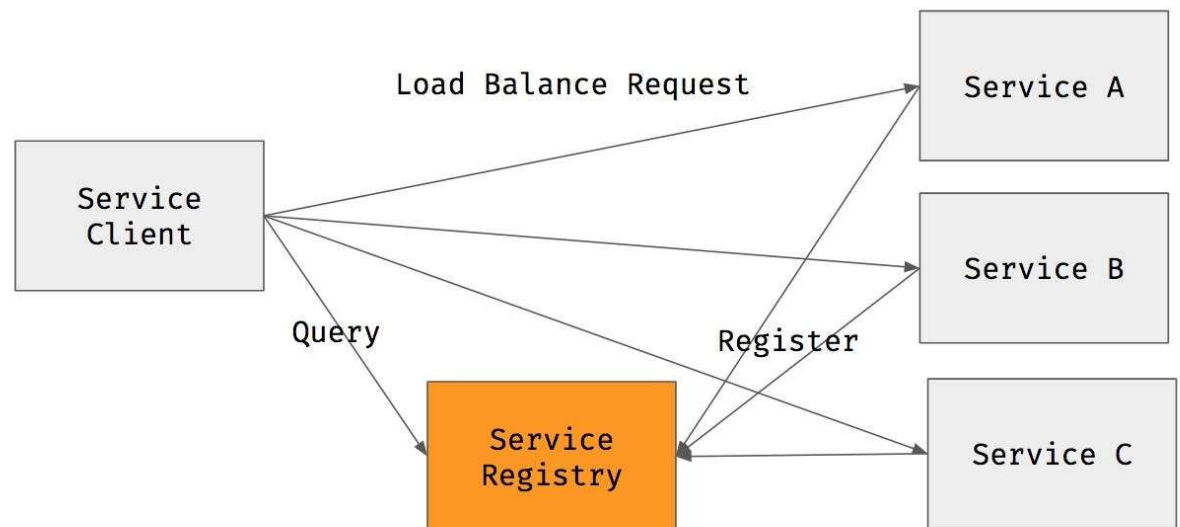
Orchestration & Automation

- ▶ Automated container deployment
- ▶ Auto-scaling based on load
- ▶ Rolling updates and rollbacks
- ▶ Configuration management
- ▶ High availability



Service Discovery & Load Balancing

- ▶ Automatic service discovery
- ▶ Built-in load balancing
- ▶ Traffic routed to healthy pods
- ▶ Improves availability
- ▶ No manual networking needed



Self-Healing

- ▶ Restarts failed containers
- ▶ Replaces unhealthy pods
- ▶ Reschedules on node failure
- ▶ Maintains desired state
- ▶ Critical for production systems